

RELATORIO TECNICO

Auditoria de Seguranca

Analise de vulnerabilidades em 5 categorias
aplicacao ImobSync

Stack: Next.js 16 + Supabase + Mercado Pago

Escopo: 62 rotas de API, schemas RLS, frontend

Metodologia: revisao linha a linha com evidencia verificada

Data: 29 de agosto de 2026

Versao: 1.0

Classificacao: Confidencial

Sumario

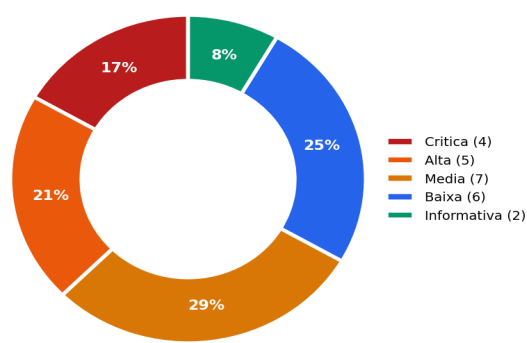
1. Resumo Executivo	2
2. Pontos Fortes e Pontos Fracos	2
2.1 Pontos Fortes	2
2.2 Pontos Fracos Centrais	3
3. Tabela de Achados Detalhada	3
CAT 1 - Banco sem Tranca (RLS)	3
CAT 2 - Permissao no Navegador	4
CAT 3 - IDOR (Insecure Direct Object Reference)	4
CAT 4 - Chaves Expostas	4
CAT 5 - Inputs sem Tratamento (XSS)	4
4. Recomendacoes Priorizadas	5
5. Issues para o GitHub	6

1. Resumo Executivo

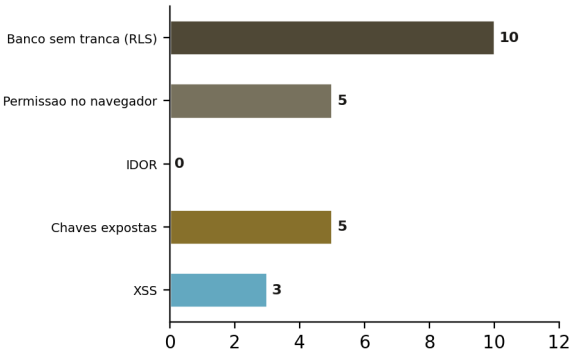
Esta auditoria avaliou a segurança da aplicação ImobSync, uma plataforma de gestão imobiliária construída com Next.js 16, Supabase (PostgreSQL com RLS) e integração com Mercado Pago para pagamentos recorrentes. O escopo abrangeu cinco categorias de vulnerabilidade: isolamento de banco de dados (RLS), verificação de permissões no backend, IDOR, exposição de chaves e tratamento de entradas (XSS). Foram analisadas sistematicamente todas as 62 rotas de API, 4 arquivos de schema SQL, 3 arquivos de guardas de autenticação e todo o código-fonte do frontend relacionado a renderização de dados de usuários. O mapeamento de cada categoria para a stack do projeto foi: RLS do Supabase para isolamento de dados, requireAdminSistema() e requireWriteAccess() para permissões, filtros por user.id para IDOR, process.env sem hardcodes para chaves, e React JSX auto-escape para XSS.

Total	Criticos	Altos	Medios	Baixos	Informativos
24	4	5	7	6	2

Distribuição por Severidade



Achados por Categoria



A análise revelou **4 vulnerabilidades críticas** que exigem ação imediata: três endpoints de debug/configuração sem autenticação que permitem manipulação direta de planos de pagamento no Mercado Pago e leitura de dados sensíveis com a service_role key, além de RLS desabilitado em tabelas de analytics que expõe dados de usuários via API REST do Supabase. Também foram identificados 5 achados de severidade alta, 7 médios e 6 baixos. A aplicação demonstrou fortalezas significativas em várias áreas, especialmente a ausência total de vulnerabilidades IDOR e a verificação HMAC correta no webhook de pagamento. A categoria IDOR retornou zero achados, o que indica um modelo de autorização bem implementado nas operações CRUD.

2. Pontos Fortes e Pontos Fracos

2.1 Pontos Fortes

- Zero IDOR:** Todas as 62 rotas de API foram auditadas sistematicamente. Nenhuma vulnerabilidade IDOR encontrada. Todas as rotas que acessam objetos por ID verificam posse via user.id da sessão ou requerem a função requireAdminSistema(). Este é um ponto forte significativo.
- HMAC no Webhook:** O endpoint /api/webhooks/mercadopago verifica a assinatura x-signature via HMAC-SHA256 antes de processar qualquer notificação de pagamento, impedindo notificações forjadas.
- Proteção CAS em Assinaturas:** As rotas de cancelamento de assinatura usam Compare-And-Swap com a cláusula .eq("status", "active") para prevenir condições de corrida durante cancelamentos simultâneos.
- Validação de Admin Consistente:** Todas as 20+ rotas sob /api/admin-sistema/ validam requireAdminSistema() antes de qualquer operação, exceto os 3 endpoints de debug identificados nesta auditoria.

Expiracao Preguicosa de Assinaturas: O subscription-guard.ts expira automaticamente assinaturas vencidas mesmo que o cron falhe, garantindo que acessos expirados sejam bloqueados na proxima verificacao.

Sem XSS no Frontend React: Nenhum uso de dangerouslySetInnerHTML com dados de usuarios. O unico uso esta em chart.tsx com dados estaticos. Todos os dados de usuarios sao renderizados via JSX que auto-escapa, e nenhum eval/new Function/innerHTML foi encontrado.

Analytics Nao Vaza Erros: O endpoint de tracking sempre retorna {ok:true} sem vaziar detalhes de erros, evitando informacao discriminante.

2.2 Pontos Fracos Centrais

Risco 1 - Endpoints de debug expostos: Tres endpoints acessiveis sem autenticacao permitem ler dados sensiveis (planos, precos) e modificar planos de pagamento no Mercado Pago. O pior caso (/api/debug/mp-test) usa a service_role key para alterar URLs de callback, podendo redirecionar pagamentos de todos os clientes para uma URL controlada pelo atacante. A correcao e trivial: adicionar requireAdminSistema() ou excluir os endpoints que ja estavam marcados para remocao com comentarios "REMOVED" apos confirmar que tudo funciona".

Risco 2 - RLS com buracos criticos: A tabela units possui uma politica UPDATE cujo nome diz "Apenas admin pode editar" mas cuja implementacao concede permissao a qualquer usuario autenticado (auth.role() = authenticated). Alem disso, 13 tabelas criadas via dashboard do Supabase nao possuem schema versionado, tornando suas politicas RLS inauditeaveis no codigo. A combinacao da anon key publica (exposta no bundle JS) com RLS ausente ou desabilitado cria um vetor de acesso direto ao banco que bypassa toda a logica de aplicacao.

Risco 3 - Limitacoes inerentes ao serverless: O armazenamento de desafios WebAuthn em memoria (Map) e o rate limiting por instancia sao ineficazes no Vercel serverless, onde cada cold start cria uma nova instancia. A funcao coordenador-access.ts adota design fail-open: se a tabela de mapeamento nao existir, acesso total e concedido por default, quando deveria falhar fechado para proteger os dados.

3. Tabela de Achados Detalhada

A tabela abaixo lista todos os 24 achados identificados, organizados por categoria. Achados marcados como Informativa nao constituem vulnerabilidades exploraveis mas representam observacoes relevantes para auditorias futuras. A coluna Arquivo:Linha indica a localizacao exata no codigo-fonte. Para cada achado, a descricao inclui o contexto de explorabilidade e o impacto potencial.

CAT 1 - Banco sem Tranca (RLS)

Severidade	Arquivo:Linha	Descricao
Critico	supabase/functions/monitoring.sql:22-23	RLS desabilitado no banco de dados. Com a anon key publica exposta no bundle, qualquer pessoa pode ler todos os eventos de usuarios (IPs, acoes, padroes de navegacao) e inserir dados falsos via API REST do Supabase, distorcendo metricas.
Critico	api/debug/mp-test.ts:13	GET /api/debug/mp-test/:id permite que qualquer usuario autenticado use o adminClient() (service_role) para ler todos os planos ativos e modificar o back_url no Mercado Pago, permitindo a um atacante redirecionar callbacks de pagamento.
Alta	supabase/schema.sql:41-42	Politica UPDATE de units com nome "Apenas admin pode editar" mas que permite que QUALQUER usuario logado modifique unidades via Supabase REST API. O nome da politica diz "Apenas admin" mas a logica nao verifica admin.
Alta	supabase/schema-admin.sql:143-148	Politica de acesso a unidades por empreendimento. Coordenador do Empreendimento A pode modificar unidades de B diretamente via REST API do Supabase, bypassando a logica de coordenador-access.ts.
Alta	supabase/schema-admin.sql:60	Politica de politica UPDATE de must_change_password. O fluxo first-login/change-password usa createClient() (anon) e o UPDATE falha silenciosamente. Flags must_change_password e must_setup_mfa nunca sao limpos para nao-admins.
Alta	lib/supabase/permissions.ts:68	Coordenadores de unidades TOPOLOGIA de validacao de assinatura (tratados como admin). Se coordenadores deveriam requerer assinatura ativa, isto e um bypass de logica de negocio que permite acesso gratuito.

Severidade	Arquivo:Linha	Descricao
ALTA	email.ts:38	displayNames do usuário inicializados com escape de HTML do email MFA. A função escapeHtml() existe (linha 67) mas não é aplicada a displayName. Usuário pode definir display_name com HTML malicioso.
CRITICA	next.config.ts:34	CSP permite desabilitar Content Security Policy. Anula proteção contra XSS. Se um vetor de injeção for encontrado, o CSP não bloqueará execução.
CRITICA	package.json:10	Permite desabilitar Content Security Policy (mitigate-html). Sem proteção para cenários futuros de renderização de HTML de usuários (descrições, emails).

4. Recomendacoes Priorizadas

CRITICA P1 - Excluir 3 endpoints de debug sem autenticação

Remover /api/debug/mp-test, /api/admin-sistema/planos/debug-mp-plan e update-mp-pix. Se necessário, adicionar requireAdminSistema(). Esses endpoints permitem manipulação real de planos de pagamento e leitura de dados com service_role key.

CRITICA P1 - Reabilitar RLS em analytics_events e unit_status_history

Criar políticas: INSERT para autenticados, SELECT para admin_sistema. Os inserts via API já usam admin client (bypassa RLS), então a mudança não afeta o tracking. A API REST direta com anon key será bloqueada.

ALTA P1 - Corrigir política UPDATE da tabela units

Alterar USING de auth.role()=authenticated para subquery verificando profiles.role=admin_sistema. A política tem nome enganoso que mascara o bug.

ALTA P1 - Adicionar isolamento por empreendimento para coordenadores

Atualizar RLS projeto_units_coordenador com JOIN em coordenador_empreendimentos para garantir que cada coordenador só modifique unidades de seus empreendimentos atribuídos.

ALTA P1 - Criar política profiles_update_own

Adicionar FOR UPDATE USING (auth.uid()=id) WITH CHECK (auth.uid()=id) para que o fluxo first-login/change-password funcione para usuários comuns (não-admins).

ALTA P1 - Escapar displayName no template de email MFA

Aplicar escapeHtml(data.displayName) em email.ts:38, consistente com o tratamento já dado ao userAgent na linha 51.

MEDIA P2 - Exportar schema completo e auditar RLS de 13 tabelas

Executar supabase db dump, commitar schemas como migrações SQL. Documentar políticas RLS de cada tabela e corrigir qualquer ausência.

MEDIA P2 - Corrigir design fail-open em coordinator-access.ts

Alterar retorno de null (acesso concedido) para [] (acesso negado) quando a tabela coordenador_empreendimentos não existir. Segurança deve falhar fechado.

MEDIA P2 - Restringir CSP: remover unsafe-inline e unsafe-eval

Migrar para nonces ou hashes no next.config.ts. Verificar se deps exigem unsafe-eval (Tailwind v4 não exige). Usar script-src-elem para separação.

MEDIA P2 - Adicionar DOMPurify e remover email admin do frontend

Instalar biblioteca de sanitizacao para uso em templates de email e cenarios futuros. Mover verificacao de admin para server-side, remover emails hardcoded do bundle.

BAIXA P3 - Limpeza: remover fallback ADMIN_EMAILS e arquivos obsoletos

Remover fallback de api-auth.ts, corrigir timing-unsafe em record-usage, excluir db.ts morto e adicionar imobsync/ ao .gitignore.

5. Issues para o GitHub

Foram geradas 6 issues prontas para copiar e colar no GitHub, agrupando os 24 achados em tickets acionaveis. As issues estao no arquivo **issues-github.md** neste mesmo diretorio (docs/security-audit/), com texto completo em Markdown incluindo titulo, labels, descricao do problema, evidencia com trecho de codigo, impacto, sugestao de correcao com exemplos, e checklist de aceite verificavel.

Resumo das issues geradas:

Issue 1 [critica]: Remover 3 endpoints de debug sem autenticacao que expoe service_role e Mercado Pago. Abrange os achados F-02, C2-01a, C2-01b e SEC-4.01.

Issue 2 [critica]: Reabilitar RLS nas tabelas analytics_events e unit_status_history. Abrange o achado F-01.

Issue 3 [alta]: Corrigir politica RLS UPDATE da tabela units e adicionar isolamento de coordenadores por empreendimento. Abrange os achados F-04 e F-05.

Issue 4 [alta]: Criar politica profiles_update_own e escapar displayName no email MFA. Abrange os achados F-06 e XSS-5.01.

Issue 5 [media]: Exportar schema completo, corrigir fail-open, restringir CSP e remover emails hardcoded. Abrange F-03, F-08, SEC-4.02, SEC-4.03, XSS-5.02 e XSS-5.03.

Issue 6 [baixa]: Limpeza geral - remover fallback ADMIN_EMAILS, corrigir timing-unsafe, excluir db.ts e diretorio imobsync. Abrange C2-03, C2-04, SEC-4.04, SEC-4.05 e SEC-4.06.